UNIVERSIDAD NACIONAL DEL ALTIPLANO

FACULTAD DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA

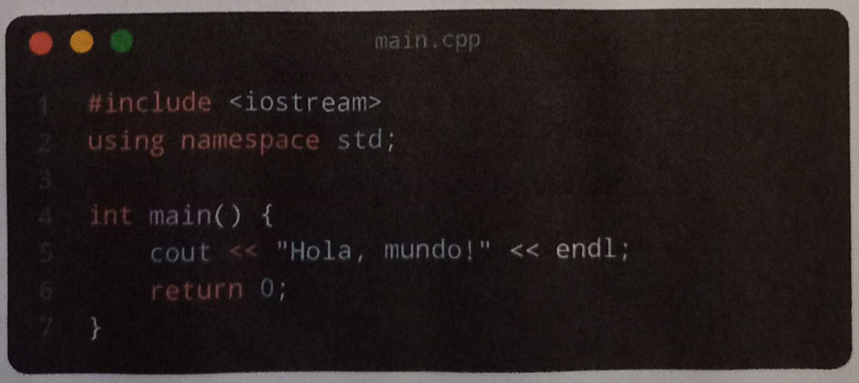
ESCUELA PROFESIONAL DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA

LENGUAJE DE PROGRAMACIÓN

CURSO: Lenguaje de Programación I

DOCENTE: Coyla Idme, Leonel

ESTUDIANTE: Mayta Ccaso, Carlos Lizandro



Puno – Perú

2026

1 Lenguaje de programación

Definición

Es un conjunto de reglas sintácticas, semánticas y símbolos formales que permite a un ser humano escribir instrucciones estructuradas para que una computadora las interprete, procese y ejecute.

1.1 Elementos esenciales de un lenguaje de programación

Sintaxis: Es el conjunto de reglas gramaticales que dictan cómo deben escribirse las palabras, símbolos y estructuras para que el código sea válido.

Semántica: Es el significado o la acción lógica que realiza cada instrucción escrita.

Vocabulario / palabras reservadas: Palabras clave con significado especial para el lenguaje que no pueden usarse para otros fines. **Ejemplo:** `print`, `cout`, `while`, `return`, etc.

Programación: Es el proceso de diseñar, escribir, probar y mantener código fuente para dar instrucciones a una computadora, permitiéndole ejecutar tareas específicas o resolver problemas de forma automatizada.

Algoritmo: Es una secuencia finita, ordenada y lógica de pasos o instrucciones precisas que deben seguirse para realizar una tarea o resolver un problema concreto.

1.2 Lenguaje de alto nivel

Es un lenguaje diseñado para ser leído y escrito fácilmente por los seres humanos. Utiliza palabras en inglés y estructuras lógicas cercanas al lenguaje nativo, independizando al programador de la arquitectura interna del hardware. **Ejemplo:** C, C++, Python, JavaScript, C#.

1.3 Lenguaje de bajo nivel

Es un lenguaje que opera con mínima o nula abstracción del hardware, diseñado para comunicarse e interactuar directamente con la memoria y el procesador de la computadora. **Ejemplo:** lenguaje máquina y lenguaje ensamblador.

2 Entorno de Desarrollo Integrado (IDE)

Es una aplicación de software que reúne todas las herramientas necesarias para que un desarrollador escriba y pruebe código en un solo lugar.

2.1 Componentes principales de un IDE

Editor de código fuente: Incluye resaltado de sintaxis, autocompletado y detección de errores en tiempo real.

Compilador / intérprete integrado: Permite ejecutar el programa con un solo clic.

Depurador (Debugger): Herramienta para inspeccionar la memoria, avanzar línea por línea y rastrear errores.

Gestor de proyectos: Organiza múltiples archivos, bibliotecas y configuraciones del programa.

3 Variables, constantes y tipos de datos

Variable

Una variable es un espacio reservado en la memoria RAM del ordenador que se utiliza para almacenar un valor que puede cambiar a lo largo de la ejecución del programa. Tiene un nombre (identificador) y un tipo de dato asociado.

Ejemplo:

- `Num1 = 5  num2 = 10  base = 10`
- Convertir grados Celsius a grados Fahrenheit: $°F = (°C \times 1{,}8) + 32$

Constante

Una constante es un valor almacenado en memoria que no puede cambiar ni ser modificado una vez que ha sido definido durante la ejecución del programa. Se utilizan para evitar errores y representar valores fijos del dominio.

Ejemplo: `PI = 3.1416`

3.1 Tipos de datos

Categoría	Tipo de dato	Tamaño habitual	Rango (aproximado)
Entero	`int`	4 bytes	-2 147 483 648 a 2 147 483 647
	`short`	2 bytes	-32 768 a 32 767
	`long`	4 u 8 bytes	-2 147 483 648 a 2 147 483 647 (en 4 bytes); -9 223 372 036 854 775 808 a 9 223 372 036 854 775 807 (en 8 bytes)
	`long long`	8 bytes	-9 223 372 036 854 775 808 a 9 223 372 036 854 775 807
Punto flotante	`float`	4 bytes	$\pm 3{,}4 \times 10^{-38}$ a $\pm 3{,}4 \times 10^{38}$
	`double`	8 bytes	$\pm 1{,}7 \times 10^{-308}$ a $\pm 1{,}7 \times 10^{308}$
Caracteres	`char`	1 byte	-128 a 127
	`wchar_t`	2 o 4 bytes	0 a 65 535 / 0 a 4 294 967 295 (depende del sistema)
Lógicos	`bool`	1 byte (típico)	0 o 1 (`true` o `false`)

4 DFD (Diagrama de Flujo de Datos)

Es una representación gráfica que muestra cómo fluye la información a través de un sistema o proceso.

4.1 Elementos principales

Símbolo	Nombre	Función
Cuadro / Rectángulo	Entidad externa	Representa la fuente u origen de los datos
Círculo / Óvalo	Proceso	Operación o función que transforma los datos
Líneas horizontales / Rectángulo abierto	Almacén de datos	Lugar donde se guardan los datos temporal o permanentemente
Flecha	Flujo de datos	Muestra la dirección en la que viaja la información entre los elementos

5 PSeInt

Es una herramienta educativa de software libre diseñada para ayudar a estudiantes a dar sus primeros pasos en la programación y el desarrollo de la lógica algorítmica.

Utiliza pseudocódigo en español, lo que permite centrarse en los conceptos fundamentales de la programación (variables, condicionales, bucles y funciones) sin la complicación de la sintaxis estricta de un lenguaje real (como C++, Java o Python).

5.1 Ejercicios

Conversión de tiempo (segundos a minutos / horas): Leer un valor en segundos e imprimir a cuántos minutos y horas equivale.

Cálculo del IGV de un producto: Solicita el precio base de un producto y calcula el monto del IGV (18 %) junto con el precio total final.

Área y perímetro de un rectángulo: Pide la longitud de la base y la altura de un rectángulo para imprimir su área y perímetro.

6 Operadores

Operadores aritméticos: `+, -, *, /, %, ++, --`

Operadores relacionales: `==, !=, >, <, >=, <=`

Operadores lógicos: `&&, ||, !` (en PSeInt: `AND, O, NOT`)

7 Estructuras condicionales

7.1 If-else

Las estructuras if-else son sentencias de control condicionales que permiten tomar decisiones en el código ejecutando bloques de instrucción únicamente cuando se cumple una condición.

1. **Estructura if (condicional simple):** Evalúa una condición. Si la condición resulta verdadera (`true`), se ejecuta el código dentro del bloque. Si es falsa (`false`), el programa omite ese bloque y continúa con la siguiente instrucción.

2. **Estructura if-else (condicional doble):** Permite definir un camino alternativo. Si la condición resulta verdadera (`true`) se ejecuta el primer bloque. Si es falso (`false`), se ejecuta obligatoriamente el bloque dentro del `else`.

Ejemplo: par o impar

Otro caso con `if / else if`: clasificar un número como positivo, negativo o nulo.

7.2 Switch-case

En C++, `switch/case` es una estructura de control de selección múltiple que evalúa el valor de una variable y ejecuta un bloque de código según la coincidencia con una lista de casos enteros o caracteres.

Ejemplo:

Ejercicios

- Crear un código C++ para la clasificación de glucosa en sangre para la detección de hipoglucemia, diabetes o prediabetes.
- Completar hasta del 1 al 10 000 en el código de números a letras.

8 Estructuras repetitivas

También llamada bucle, ciclo o iteración. Es un bloque de código que permite ejecutar una o varias instrucciones de forma recurrente mientras se cumpla una determinada condición lógica.

Su propósito principal es automatizar tareas repetitivas, evitar la duplicidad de código y procesar colecciones de datos (como listas o arreglos) de manera eficiente.

Iteración: Una iteración es cada una de las repeticiones o ejecuciones individuales de un bucle. Si un bucle se ejecuta 5 veces, se dice que realiza 5 iteraciones.

Contador: Es una variable entera cuyo valor se incrementa o decrementa en una cantidad constante (generalmente de 1 en 1) en cada iteración. Se utiliza para registrar la cantidad de veces que ocurre un evento o se repite un ciclo.

Acumulador: Es una variable que almacena la suma o multiplicación sucesiva de valores variables en cada iteración. Se utiliza para obtener totales, promedios o sumatorias acumuladas.

8.1 Control de flujo iterativo

C++ proporciona sentencias especiales para alterar la ejecución normal de una estructura repetitiva:

`break`: Interrumpe y sale inmediatamente del bucle, ignorando las iteraciones restantes.

`continue`: Salta el resto de las instrucciones de la iteración actual y pasa directamente a la siguiente iteración (evaluando de nuevo la condición de incremento).

8.2 Anidamiento básico de bucles

Tabla de multiplicación

1 × 1 = 1 2 × 1 = 2 ...

$1 \times 2 = 2$	$2 \times 2 = 4$	...
⋮	⋮	
$1 \times 10 = 10$	$2 \times 10 = 20$	$10 \times 10 = 100$

8.3 Estructura repetitiva `while` (mientras)

Es un bucle o ciclo de control que ejecuta un bloque de código de forma iterativa mientras se cumpla una condición lógica determinada. A diferencia de otros bucles como `for`, el bucle `while` se utiliza principalmente cuando no se conoce con exactitud el número exacto de iteraciones que se van a realizar antes de iniciar la ejecución.

Funcionamiento básico:

1. **Evaluación:** Antes de ejecutar el código se evalúa la condición lógica.
2. **Ejecución:** Si la condición es verdadera (`true`), se ejecuta el cuerpo del bucle.
3. **Repetición:** Al terminar el bloque, se vuelve a evaluar la condición. El proceso continúa hasta que la condición pase a ser falsa (`false`).
4. **Finalización:** Si la condición es falsa desde el inicio, el bloque dentro del bucle no se ejecutará ni una sola vez.

8.4 Estructura repetitiva `do-while` (hacer-mientras)

Es un bucle o ciclo de control que ejecuta un bloque de código al menos una vez y luego continúa repitiéndolo mientras se cumpla una condición lógica determinada. A diferencia del bucle `while` tradicional, la condición de evaluación se encuentra al final del código.

Funcionamiento básico:

1. **Ejecución obligatoria:** El programa entra al ciclo y ejecuta las instrucciones del bloque directamente, sin comprobar ninguna condición previa.
2. **Evaluación posterior:** Al llegar al final del bloque, se evalúa la condición lógica expresada en el `while`.
3. **Repetición:** Si la condición es verdadera (`true`), el flujo regresa al inicio del bloque para repetir el proceso.
4. **Finalización:** Si la condición es falsa (`false`), el ciclo termina y el programa continúa con la instrucción siguiente.

Ejercicios

- Leer *n* números enteros, obtener la suma total y el promedio de los números ingresados.
- Crear la tabla de multiplicación del 7.

9 Arreglos

Un arreglo (también conocido como array, vector o matriz) es una estructura de datos homogénea y de tamaño fijo que permite almacenar una colección de elementos del mismo tipo bajo un único nombre. Cada elemento individual dentro del arreglo se identifica y se accede mediante uno o más índices numéricos.

9.1 Arreglo unidimensional (vector)

Un arreglo unidimensional es una colección lineal de datos que se organizan de forma secuencial, uno al lado del otro en la memoria del computador. Se puede visualizar como una sola fila con múltiples casillas.

Ejemplo:

índices	0	1	2	3	4
valores	**2**	**14**	**15**	**16**	**20**

Tipo de dato: `int`

Ejercicio: Leer *n* notas, mostrar las notas ingresadas y obtener el promedio.

9.2 Arreglo bidimensional (matriz)

Un arreglo bidimensional es una estructura organizada en una tabla de dos dimensiones: filas y columnas. Se puede imaginar como una cuadrícula o una hoja de cálculo, donde la intersección de una fila y una columna determina una posición o casilla específica.

Ejemplo:

	0	1	2
0	**14**	**15**	**16**
1	**8**	**13**	**20**